Chapter 10 | Exercise solutions with Python

Incremental Decoding and KV Representations

This notebook solves Exercises 5–8. It builds causal masks from absolute positions, compares a full attention pass with chunks that reuse a cache, counts the bytes in two storage formats, and checks two algebraic orders of latent attention. Exercises 1–4 and 9–12 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch10_kv" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Build a mask for a chunk

The cache holds keys 0 and 1. Append two new keys at positions 2 and 3, one for each of the two new queries. The query rows have local indices 0 and 1 in the chunk array, but their **absolute positions**, meaning their indices in the whole sequence, are 2 and 3. Those whole-sequence indices determine which keys each query can read.

A **causal mask** is a Boolean array that allows a query to read a key only if the key is at the same position or an earlier one. `causal_mask` takes the absolute position of the first query, the number of queries, and the number of keys. Here the correct arguments are 2, 2, and 4. In both runs, columns are keys 0, 1, 2, and 3 in that order. Convert Boolean entries to integers for display so that 1 means an allowed read and 0 means a blocked read.

```
from examples.ch10_kv.model import causal_mask

correct_mask = causal_mask(2, 2, 4)
reset_mask = causal_mask(0, 2, 4)
lost_reads = correct_mask & ~reset_mask

np.testing.assert_array_equal(
    correct_mask, [[1, 1, 1, 0], [1, 1, 1, 1]]
)
np.testing.assert_array_equal(
    reset_mask, [[1, 0, 0, 0], [1, 1, 0, 0]]
)

print("Correct mask for absolute queries 2 and 3")
print(correct_mask.astype(int))
print("Mask after assigning the queries positions 0 and 1")
print(reset_mask.astype(int))

for row, abs_query in enumerate((2, 3)):
    key_ids = np.flatnonzero(lost_reads[row]).tolist()
    print(f"Actual query {abs_query} loses keys {key_ids}")
```

```
OUTPUT
Correct mask for absolute queries 2 and 3
[[1 1 1 0]
 [1 1 1 1]]
Mask after assigning the queries positions 0 and 1
[[1 0 0 0]
 [1 1 0 0]]
Actual query 2 loses keys [1, 2]
Actual query 3 loses keys [2, 3]
```

Query 2 should read keys 0, 1, and 2. The reset makes the mask treat it as query 0, so it loses keys 1 and 2. Query 3 should read all four keys. The reset makes it look like query 1, so it loses keys 2 and 3. The data can still contain the correct keys in their correct order. The mask decides which of those stored keys the new queries actually get to use.

With two sequences and four query heads, the scores have shape $[2, 4, 2, 4]$ in batch, head, query, and key order. Give the mask shape $[2, 1, 2, 4]$. Broadcasting uses the single head entry for all four heads of a sequence. The two batch entries still carry independent patterns. A mask of shape $[2, 2, 4]$ would align its leading axis with the head axis because NumPy aligns dimensions from the right.

For a concrete example, let sequence A use queries at positions 2 and 3, and sequence B use queries at positions 1 and 2. Each has four available key positions in its key array. Construct the two causal patterns, then insert an axis of length one at the head position. The broadcast view below shows how the mask aligns with the scores in `grouped_attention`.

```
pattern_a = correct_mask
pattern_b = causal_mask(1, 2, 4)
batch_mask = np.stack((pattern_a, pattern_b))[:, None, :, :]
score_mask = np.broadcast_to(batch_mask, (2, 4, 2, 4))

np.testing.assert_array_equal(score_mask[0, 3], pattern_a)
np.testing.assert_array_equal(score_mask[1, 3], pattern_b)
assert not np.array_equal(score_mask[0, 3], score_mask[1, 3])

print("Mask supplied to attention:", batch_mask.shape)
print("Mask aligned with scores:", score_mask.shape)
for sequence in range(2):
    print(f"Sequence {sequence}, head 3")
    print(score_mask[sequence, 3].astype(int))
```

```
OUTPUT
Mask supplied to attention: (2, 1, 2, 4)
Mask aligned with scores: (2, 4, 2, 4)
Sequence 0, head 3
[[1 1 1 0]
 [1 1 1 1]]
Sequence 1, head 3
[[1 1 0 0]
 [1 1 1 0]]
```

Sequence B's first query is one position earlier than A's and cannot read key 2. B's second query cannot read key 3. Within either sequence, every head receives the same permission table. If an input also had padding, intersect its causal pattern with a mask for valid keys. Every query sent to this implementation needs at least one allowed key. This example constructs a mask for attention on supplied projections. The teaching cache API assumes equally long, contiguous histories with one common chunk start.

6. Compare full and cached attention

Take the same six positions and evaluate them in two ways. The full pass supplies all six query rows at once. The chunked pass supplies the first three rows, then two more, then the last row. The input projections, head grouping, and causal rule stay fixed. This lets us isolate the effect of splitting the work into chunks.

`projected_example()` supplies already projected query, key, and value arrays. Their axes are batch, head, position, and feature. Keep four query heads but slice the **head axis** of the keys and values using `[:, :2]`. That preserves all six positions for the two retained KV heads. Query heads 0 and 1 read KV head 0. Query heads 2 and 3 read KV head 1. This is grouped-query attention. The full pass uses a six-by-six causal mask to block later keys, even though they are already present in the arrays.

The returned weights have one entry per batch item, query head, query row, and key position. The table printed below shows the weights for the first query head. Each row adds to one, and every entry for a future key is zero. The outputs have two features per head and query because the value vectors have width two.

```
from examples.ch10_kv.model import grouped_attention
from examples.ch10_kv.run_examples import projected_example

q, all_keys, all_values = projected_example()
k, v = all_keys[:, :2], all_values[:, :2]
full_mask = causal_mask(0, 6, 6)
full_out, full_weights = grouped_attention(q, k, v, full_mask)

assert q.shape == full_out.shape == (1, 4, 6, 2)
assert k.shape == v.shape == (1, 2, 6, 2)
assert full_weights.shape == (1, 4, 6, 6)
assert np.all(full_weights[..., ~full_mask] == 0)
np.testing.assert_allclose(
    full_weights.sum(axis=-1), 1, rtol=0, atol=1e-12
)

print("Query shape:", q.shape)
print("Key and value shape:", k.shape)
print("Full output shape:", full_out.shape)
print("Full weight shape:", full_weights.shape)
print("Full causal weights for head 0")
with np.printoptions(precision=6, suppress=True):
    print(full_weights[0, 0])
```

OUTPUT

```
Query shape: (1, 4, 6, 2)
Key and value shape: (1, 2, 6, 2)
Full output shape: (1, 4, 6, 2)
Full weight shape: (1, 4, 6, 6)
Full causal weights for head 0
[[1.       0.       0.       0.       0.       0.      ]
 [0.232691 0.767309 0.       0.       0.       0.      ]
 [0.274329 0.408696 0.316974 0.       0.       0.      ]
 [0.446164 0.2095   0.217144 0.127192 0.       0.      ]
 [0.212473 0.090016 0.167331 0.406598 0.123582 0.      ]
 [0.076939 0.148178 0.14785  0.253259 0.247774 0.126   ]]
```

In the chunked run, `run_chunks` calls `attention_step` once for each chunk. Each call keeps the previous keys and values, appends the new ones on the position axis, and uses the previous cache length as the absolute query start.

Chunk	Previous cache length	Absolute queries	Key positions after append
1	0	0, 1, 2	0, 1, 2
2	3	3, 4	0, 1, 2, 3, 4
3	5	5	0, 1, 2, 3, 4, 5

In the second chunk, query 3 can read keys 0 through 3 and query 4 can read keys 0 through 4. Key 4 has already been appended when query 3 runs. It still gets zero weight in that row. In the full pass the same row can see the same four keys, while its entries for keys 4 and 5 are masked. This reasoning applies to every query, so each head forms the same weighted sum of the same allowed values, up to rounding.

Concatenate the chunk outputs in position order, then compare each float64 output coordinate with the corresponding full-pass coordinate. The relative tolerance is zero. The absolute tolerance of 10^{-12} applies equally to outputs near zero and to larger outputs. The cache comparison is exact because storing or appending a projected row copies its entries without recalculating them.

```
from examples.ch10_kv.run_examples import run_chunks

chunk_sizes = (3, 2, 1)
cached_out, (cached_k, cached_v) = run_chunks(q, k, v, chunk_sizes)
np.testing.assert_allclose(
    cached_out, full_out, rtol=0, atol=1e-12
)
np.testing.assert_array_equal(cached_k, k)
np.testing.assert_array_equal(cached_v, v)

error = np.abs(cached_out - full_out)
start = 0
for size in chunk_sizes:
    end = start + size
    local_error = error[:, :, start:end].max()
    print(f"Positions {start} through {end - 1}: {local_error:.3e}")
    start = end

print(f"Maximum output difference: {error.max():.3e}")
print("Cache shapes:", cached_k.shape, cached_v.shape)
print("Final query output for each head in chunked attention")
with np.printoptions(precision=9, suppress=True):
    print(cached_out[0, :, -1, :])
```

OUTPUT

```
Positions 0 through 2: 0.000e+00
Positions 3 through 4: 0.000e+00
Positions 5 through 5: 1.665e-16
Maximum output difference: 1.665e-16
Cache shapes: (1, 2, 6, 2) (1, 2, 6, 2)
Final query output for each head in chunked attention
[[0.149017163 0.312294206]
 [0.543017043 0.847100077]
 [0.066147024 0.367019286]
 [0.353063196 0.351276429]]
```

Both output arrays have shape $[1, 4, 6, 2]$. Each cache array has shape $[1, 2, 6, 2]$ and stores exactly the projected rows for its two KV heads. The output uses four heads because the four separate queries can produce different weight patterns, even when two of them read the same history. The cache holds two heads because the history is shared within each group.

This check isolates the effect of processing the same attention inputs in chunks. Passing both computations the same projected arrays keeps any differences in upstream positional transforms or other decoder layers outside the experiment.

7. Check what a byte count means

Take the same six projected positions as in Exercise 6 and replay the chunk schedule $(3, 2, 1)$. Keep all four query heads and two key and value heads. Two query heads read each KV head. The cache therefore stores two heads per position in each of its arrays.

Each cached array has shape $[1, 2, 6, 2]$, in batch, head, position, and feature order. It contains $1 \cdot 2 \cdot 6 \cdot 2 = 24$ scalars. NumPy calls the number of bytes per scalar `itemsize`. With `float64`, each scalar uses eight bytes, so each array uses $24 \cdot 8 = 192$ bytes. The two arrays together use 384 bytes.

The helper `cache_bytes` counts the two feature widths together. Here we pass $B = 1$ batch item, $L = 1$ layer, $T = 6$ positions, $H_{KV} = 2$ KV heads, and $d_k = d_v = 2$. With s bytes per scalar, the formula is

$$BLTH_{KV}(d_k + d_v)s = 1 \cdot 1 \cdot 6 \cdot 2 \cdot (2 + 2) \cdot 8 = 384 \text{ bytes.}$$

We will compare the formula with the sum of `nbytes` from the two arrays returned by `run_chunks`. Then we will replay the same chunks after casting the keys and values to `float32`, with the queries unchanged. Each array still has 24 scalars. At four bytes each, those scalars take 96 bytes per array and 192 bytes in total.

```
from examples.ch10_kv.model import cache_bytes
from examples.ch10_kv.run_examples import projected_example, run_chunks

byte_queries, byte_keys, byte_values = projected_example()
byte_keys, byte_values = byte_keys[:, :2], byte_values[:, :2]
byte_chunks = (3, 2, 1)
byte_results = {}

for dtype in (np.float64, np.float32):
    keys = byte_keys.astype(dtype, copy=False)
    values = byte_values.astype(dtype, copy=False)
    output, (stored_keys, stored_values) = run_chunks(
        byte_queries, keys, values, byte_chunks
    )
    byte_results[dtype] = output, stored_keys, stored_values

    batch, heads, positions, key_width = stored_keys.shape
    value_width = stored_values.shape[-1]
    itemsize = stored_keys.itemsize
    counted = stored_keys.nbytes + stored_values.nbytes
    predicted = cache_bytes(
        batch, 1, positions, heads, key_width, value_width, itemsize
    )

    np.testing.assert_array_equal(stored_keys, keys)
    np.testing.assert_array_equal(stored_values, values)
    assert stored_keys.dtype == stored_values.dtype == np.dtype(dtype)
    assert output.dtype == np.dtype(np.float64)
    assert counted == predicted == 48 * itemsize

    print(f"{np.dtype(dtype).name} storage ({itemsize} bytes/scalar)")
    print(f"  keys    {stored_keys.dtype}  {stored_keys.nbytes} B")
    print(f"  values  {stored_values.dtype}  {stored_values.nbytes} B")
    print(f"  total   {counted} B (formula {predicted} B)")
    print(f"  attention output dtype {output.dtype}")
```

OUTPUT

```
float64 storage (8 bytes/scalar)
  keys    float64  192 B
  values  float64  192 B
  total   384 B (formula 384 B)
  attention output dtype float64
float32 storage (4 bytes/scalar)
  keys    float32  96 B
  values  float32  96 B
  total   192 B (formula 192 B)
  attention output dtype float64
```

Both runs produce `float64` attention output. The stored arrays keep the dtype of the rows appended to them. Inside `grouped_attention`, the queries are cast to `float64` before the score multiplication, and softmax produces `float64` weights. NumPy promotes the `float32` keys and values for those respective multiplications. For example, the first key scalar was about 0.60587896545. Storing it as `float32` rounds it to about 0.60587894917. Promoting the stored scalar to `float64` for arithmetic keeps that rounded value.

To isolate the effect of storage precision, cast the rounded keys and values back to `float64` before replaying. That replay should match the run that stored `float32`. Comparing either with the original `float64` run measures the effect of rounding the inputs for this example.

```
out64, key64, _ = byte_results[np.float64]
out32, key32, value32 = byte_results[np.float32]
promoted_output, _ = run_chunks(
    byte_queries,
    key32.astype(np.float64),
    value32.astype(np.float64),
    byte_chunks,
)

np.testing.assert_allclose(out32, promoted_output, atol=1e-12, rtol=0)
print(f"Original key scalar  {key64.flat[0]:.12f}")
print(f"Rounded key scalar   {float(key32.flat[0]):.12f}")
delta = np.max(np.abs(out64 - out32))
print(f"Max output change from rounding  {delta:.3e}")
```

```
OUTPUT
Original key scalar  0.605878965449
Rounded key scalar   0.605878949165
Max output change from rounding  5.744e-08
```

The smaller cache changes the result by about $5.74 \cdot 10^{-8}$ at the most affected output coordinate in this run. That number depends on the input and the calculation. The byte totals are exact. `nbytes` measures the payload of the two arrays. It leaves out the queries, attention scores and weights, allocations used when appending a chunk, and the surrounding Python objects.

8. Compare two orders of latent attention

Imagine one query reading two earlier positions. In the expanded calculation, we apply each head's key and value matrices to both cached rows first. In the latent calculation, each head transforms its content query once, compares it with the shared latent rows, and mixes those rows before applying the value matrix. The rotary part of each score uses the same positional vectors in both runs.

`latent_inputs()` supplies six arrays in the order expected by both functions. They are the content queries, rotary queries, latent history, rotary key history, key projections, and value projections. The `*inputs` syntax passes them as six positional arguments. The example gives one query per head for two heads. Each output row contains two value coordinates, and each weight row contains two probabilities, one per cached position. We compare both arrays at absolute tolerance 10^{-12} and zero relative tolerance.

```
from examples.ch10_kv.latent import (
    expanded_attention, latent_attention, latent_inputs,
)

inputs = latent_inputs()
expanded_out, expanded_w = expanded_attention(*inputs)
latent_out, latent_w = latent_attention(*inputs)

np.testing.assert_allclose(expanded_w, latent_w, atol=1e-12, rtol=0)
np.testing.assert_allclose(expanded_out, latent_out, atol=1e-12, rtol=0)

with np.printoptions(precision=8):
    print("Weights (expanded, then latent)")
    print(expanded_w)
    print(latent_w)
    print("Outputs (expanded, then latent)")
    print(expanded_out)
    print(latent_out)
weight_delta = np.max(np.abs(expanded_w - latent_w))
output_delta = np.max(np.abs(expanded_out - latent_out))
print(f"Max weight delta {weight_delta:.3e}")
print(f"Max output delta {output_delta:.3e}")
```

OUTPUT

```
Weights (expanded, then latent)
[[0.81757448 0.18242552]
 [0.88079708 0.11920292]]
[[0.81757448 0.18242552]
 [0.88079708 0.11920292]]
Outputs (expanded, then latent)
[[1.18242552 3.2702979 ]
 [3.52318831 1.11920292]]
[[1.18242552 3.2702979 ]
 [3.52318831 1.11920292]]
Max weight delta 0.000e+00
Max output delta 0.000e+00
```

Reconstruct the scores for the first head, which is at index 0 in the arrays. Its queries are $q^C = (2, 1)$ for content and $q^R = (0, 1)$ for the rotary part. The content key matrix is $W_{UK} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$. Applying it to the two latent rows gives expanded content keys $(1, 2)W_{UK} = (3, 2)$ and $(2, 0)W_{UK} = (2, 0)$.

Contribution	Position 0	Position 1
Content $q^C \cdot k^C$	$2 \cdot 3 + 1 \cdot 2 = 8$	$2 \cdot 2 + 1 \cdot 0 = 4$
Rotary $q^R \cdot k^R$	$(0, 1) \cdot (1, 0) = 0$	$(0, 1) \cdot (0, 1) = 1$
Sum before scaling	8	5
Score after dividing by $\sqrt{2+2} = 2$	4	2.5

In the latent calculation, transform the content query as $\bar{q}^C = q^C W_{UK}^\top = (2, 3)$. Its dot products with $(1, 2)$ and $(2, 0)$ are still 8 and 4. This follows by writing the expanded product as $q^C(cW_{UK})^\top = (q^C W_{UK}^\top)c^\top$ for any row c. The rotary scores come from a separate dot product. The scale stays $\sqrt{d_h + d_R}$, where d_h is the expanded content width and d_R is the rotary width. Moving the key projection leaves the score represented by the dot product unchanged. In this example the latent and content widths happen to be equal.

```
content_q, rotary_q, latent_rows, rotary_keys, key_maps, value_maps = (
    inputs
)
first_content = content_q[0]
first_rotary = rotary_q[0]
first_key_map = key_maps[0]
expanded_keys = latent_rows @ first_key_map
absorbed_query = first_content @ first_key_map.T

expanded_terms = first_content @ expanded_keys.T
absorbed_terms = absorbed_query @ latent_rows.T
rotary_terms = first_rotary @ rotary_keys.T
scale = np.sqrt(content_q.shape[-1] + rotary_q.shape[-1])
scaled_scores = (absorbed_terms + rotary_terms) / scale

np.testing.assert_array_equal(expanded_terms, [8, 4])
np.testing.assert_array_equal(absorbed_terms, expanded_terms)
np.testing.assert_array_equal(rotary_terms, [0, 1])
np.testing.assert_array_equal(scaled_scores, [4, 2.5])
print("Content from expanded keys", expanded_terms)
print("Content from latent rows  ", absorbed_terms)
print("Rotary contribution       ", rotary_terms)
print("Scaled scores             ", scaled_scores)
```

```
OUTPUT
Content from expanded keys [8. 4.]
Content from latent rows   [8. 4.]
Rotary contribution        [0. 1.]
Scaled scores              [4.  2.5]
```

Softmax of $(4, 2.5)$ gives first-head weights approximately $(0.81757448, 0.18242552)$. For example, the first probability is $e^4/(e^4 + e^{2.5}) = 1/(1 + e^{-1.5})$. Write these weights as a and b, so $a + b = 1$. The first head's value matrix is $\operatorname{diag}(1, 2)$, which sends the two latent rows to $(1, 4)$ and $(2, 0)$. Weighting these gives $(a + 2b, 4a)$. If we mix the latent rows first, we get $(a + 2b, 2a)$, which the value matrix again maps to $(a + 2b, 4a)$. A fixed linear projection distributes over the weighted sum.

The history consists of the third and fourth input arrays, which are `inputs[2]` and `inputs[3]` in Python. Each has two rows with two coordinates, so each has four scalars. Both heads share those histories. The sum is eight scalars. If we kept separate expanded content keys and values for both heads, that would take $2 \cdot 2 \cdot (2 + 2) = 16$ scalars, with another four for the shared rotary keys. This comparison counts the stored history. The current query and the learned projection matrices are separate.

```
first_exp = np.exp(scaled_scores - scaled_scores.max())
first_weights = first_exp / first_exp.sum()
first_values = latent_rows @ value_maps[0]
weighted_values = first_weights @ first_values
projected_mixture = (first_weights @ latent_rows) @ value_maps[0]

np.testing.assert_allclose(first_weights, latent_w[0], atol=1e-12,
                           rtol=0)
np.testing.assert_allclose(weighted_values, latent_out[0], atol=1e-12,
                           rtol=0)
np.testing.assert_allclose(weighted_values, projected_mixture,
                           atol=1e-12, rtol=0)
with np.printoptions(precision=8):
    print("First-head weights", first_weights)
    print("Expand then mix   ", weighted_values)
    print("Mix then expand   ", projected_mixture)

history_scalars = inputs[2].size + inputs[3].size
assert history_scalars == 8
print("Latent history", inputs[2].shape, inputs[2].size, "scalars")
print("Rotary history", inputs[3].shape, inputs[3].size, "scalars")
print("Total history ", history_scalars, "scalars")
```

OUTPUT

```
First-head weights [0.81757448 0.18242552]
Expand then mix    [1.18242552 3.2702979 ]
Mix then expand    [1.18242552 3.2702979 ]
Latent history (2, 2) 4 scalars
Rotary history (2, 2) 4 scalars
Total history  8 scalars
```